Jawaharlal Nehru Technological University Hyderabad
University College of Engineering, Science & Technology

Real-Time Research Project Report

On

ResumeIQ: Semantic AI Resume Screening and Candidate Ranking System

Submitted By

A. SATHVIK - 23011M2110
SHIVA SATVIK JAKKAM - 23011M2111
CH. GANESH - 23011M2207

Department of Computer Science and Engineering
JNTUH University College of Engineering, Science & Technology
Kukatpally, Hyderabad - 500 085

Academic Year 2025-2026

DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled ResumeIQ: Semantic AI Resume Screening and Candidate Ranking System is prepared as a record of project work carried out by us. The system, documentation, testing notes, and explanation are based on our implementation of a local semantic AI resume screening application using Flask, SQLite, NLP preprocessing, resume parsing, sentence embeddings, ranking logic, and recruiter analytics. The material presented in this report is written for academic submission and demonstration purposes.

Date: _____

Signature: _____

CERTIFICATE BY THE SUPERVISOR

This is to certify that the project report entitled ResumeIQ: Semantic AI Resume Screening and Candidate Ranking System is submitted by A. Sathvik, Shiva Satvik Jakkam, and Ch. Ganesh in partial fulfillment of academic project requirements. The work describes a practical recruitment assistance system with local semantic AI matching, candidate ranking, and analytics for recruiter decision support.

Date: _____

Signature: _____

CERTIFICATE BY THE HEAD OF THE DEPARTMENT

This is to certify that the project work entitled ResumeIQ: Semantic AI Resume Screening and Candidate Ranking System has been prepared under the Department of Computer Science and Engineering. The report explains the problem, system design, implementation, testing, results, and future scope of the project.

Date: _____

Signature: _____

ABSTRACT

In the current recruitment environment, organizations receive a large number of resumes for each job opening, making manual screening slow, repetitive, and vulnerable to inconsistency. ResumeIQ addresses this problem by providing a lightweight semantic AI recruiter assistance system that uploads resumes, extracts resume text, preprocesses content, identifies skills, compares each resume with a job requirement, ranks candidates, and produces recruiter-focused analytics from the uploaded resume set.

The application uses Flask for the backend, SQLite for storage, pdfplumber and python-docx for resume parsing, NLTK for text preprocessing, Sentence Transformers with the all-MiniLM-L6-v2 model for semantic embeddings, and cosine similarity for meaning-level comparison. Unlike keyword-only screening, the system can recognize conceptual similarity between expressions such as backend API development and building REST APIs with FastAPI. The ranking mechanism combines semantic relevance, skill overlap, experience relevance, and project relevance into a clear final match percentage.

ResumeIQ also includes a modern recruiter dashboard built with Flask templates, TailwindCSS, and Chart.js. The dashboard displays match distribution, semantic relevance distribution, top detected skills, domain distribution, shortlisted versus rejected counts, skill gaps, resume completeness, and an AI insight panel. Recruiters can create jobs, expand job details, upload multiple resumes, filter candidates, shortlist or reject candidates, and download a PDF report of shortlisted candidates. The system is designed to run locally, remain modular, and be easy to explain during academic evaluation.

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, faculty members, and the Department of Computer Science and Engineering for their guidance and support throughout the development of ResumeIQ. Their direction helped us refine the idea from a simple resume ranking tool into a more complete recruiter assistance system with parsing, semantic matching, skill analytics, and an explainable dashboard.

We also acknowledge the open-source communities behind Flask, SQLite, NLTK, Sentence Transformers, pdfplumber, python-docx, TailwindCSS, Chart.js, NumPy, pandas, scikit-learn, and ReportLab. These technologies made it possible to build a practical and locally executable system with modern NLP and semantic matching capabilities.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

Recruiters often receive many resumes for a single job opening, and manual screening becomes slow when every candidate must be compared against the same requirement. The difficulty increases when resumes are written in different formats, use different section headings, or describe similar experience with different words. A backend candidate may write that they built REST APIs with FastAPI, while a job description may ask for backend API development. A keyword-only system may fail to connect these two statements even though they describe closely related work.

ResumeIQ solves this problem by combining explicit skill matching with semantic similarity. It does not only count whether exact words appear; it also compares the meaning of the resume and the job description through sentence embeddings. This provides a better first-level ranking for the recruiter and reduces the amount of repetitive reading required before identifying promising candidates.

Another problem in recruitment is the lack of clear analytics. A recruiter needs to know whether the applicant pool is strong, which skills are common, which skills are missing, and how many candidates are shortlisted or rejected. ResumeIQ therefore treats analytics as an important part of the screening system rather than a decoration.

The problem is not limited to large companies. Colleges, startups, placement teams, and small HR departments may also receive many resumes but may not have access to expensive applicant tracking systems. A local, lightweight, and explainable system can help these users evaluate resumes with less manual effort while still keeping the recruiter in control of the final decision.

The recruitment domain is suitable for this project because it naturally contains unstructured text, repeated decision-making, and the need for transparent comparison. Resumes are not written in one fixed format, so the system must be able to handle variation in wording, layout, and skill expression. This creates a meaningful problem for NLP and semantic AI.

ResumeIQ also focuses on the recruiter experience. A recruiter does not want to inspect embeddings or raw vectors; the recruiter wants to know who is strongest, which skills matched, which skills are missing, and whether the overall applicant pool is suitable for the role. Therefore the project converts technical analysis into clear percentages, badges, tables, charts, and insights.

The project is intentionally scoped so that it can be demonstrated end to end. A user can create a job, upload resumes, review rankings, shortlist candidates, and download a PDF report within one session. This makes the implementation practical for presentation and also proves that the individual modules are connected properly.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

1.2 Motivation

The motivation for this project comes from the need to demonstrate a practical use of NLP and semantic AI in a real-world recruitment problem. Resume screening is a familiar and useful domain because the input documents are text-heavy, the decision depends on meaning, and the result must be understandable to a human recruiter. A system that can read resumes, compare them with job descriptions, and explain ranking through scores and skill gaps is both academically relevant and practically useful.

The project was designed to remain lightweight and locally executable. Instead of using cloud AI services or a complicated deployment architecture, ResumeIQ uses Python, Flask, SQLite, NLTK, Sentence Transformers, and Chart.js. This makes the project suitable for a student machine and easy to present during a viva. The all-MiniLM-L6-v2 model was selected because it provides strong semantic embeddings while remaining small enough for CPU execution.

ResumeIQ is also motivated by explainability. Recruiters should not receive only a mysterious final result. They should see match percentage, semantic percentage, skill match percentage, experience relevance, resume strength, overlap skills, missing skills, candidate status, and dashboard-level insights. This makes the system a recruiter assistance tool rather than an opaque automatic decision maker.

The project further motivates students to understand how different AI and software engineering concepts work together. ResumeIQ connects document parsing, NLP preprocessing, vector embeddings, cosine similarity, database design, web development, visualization, testing, and report generation into one complete workflow.

The recruitment domain is suitable for this project because it naturally contains unstructured text, repeated decision-making, and the need for transparent comparison. Resumes are not written in one fixed format, so the system must be able to handle variation in wording, layout, and skill expression. This creates a meaningful problem for NLP and semantic AI.

ResumeIQ also focuses on the recruiter experience. A recruiter does not want to inspect embeddings or raw vectors; the recruiter wants to know who is strongest, which skills matched, which skills are missing, and whether the overall applicant pool is suitable for the role. Therefore the project converts technical analysis into clear percentages, badges, tables, charts, and insights.

The project is intentionally scoped so that it can be demonstrated end to end. A user can create a job, upload resumes, review rankings, shortlist candidates, and download a PDF report within one session. This makes the implementation practical for presentation and also proves that the individual modules are connected properly.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

1.3 Objectives and Scope

The primary objective of ResumeIQ is to build a complete semantic AI resume screening system that can upload multiple resumes, parse resume content, preprocess text, extract skills, compare resumes with job descriptions, generate similarity scores, rank candidates, shortlist suitable candidates, and generate recruiter analytics. The application must support the full flow from recruiter login to shortlisted candidate PDF export.

The second objective is to demonstrate semantic AI without unnecessary complexity. The system uses sentence embeddings and cosine similarity to compare meaning, but the architecture remains simple. Each module has a clear responsibility: parser, preprocessing, skills, embeddings, matching, ranking, analytics, database, and templates. This modular design makes the system easier to debug and explain.

The scope of ResumeIQ is limited to recruiter/admin-side screening. The system does not include a candidate portal, online application form, email communication, interview scheduling, or enterprise applicant tracking workflows. This boundary keeps the project focused on resume analysis and candidate ranking, which are the core requirements of the system.

The project also includes 100 generated visual PDF resumes across many domains so that testing and demonstrations can be performed without private candidate data. These resumes help show ranking behavior, domain distribution, and skill-gap analytics in a realistic way.

The recruitment domain is suitable for this project because it naturally contains unstructured text, repeated decision-making, and the need for transparent comparison. Resumes are not written in one fixed format, so the system must be able to handle variation in wording, layout, and skill expression. This creates a meaningful problem for NLP and semantic AI.

ResumeIQ also focuses on the recruiter experience. A recruiter does not want to inspect embeddings or raw vectors; the recruiter wants to know who is strongest, which skills matched, which skills are missing, and whether the overall applicant pool is suitable for the role. Therefore the project converts technical analysis into clear percentages, badges, tables, charts, and insights.

The project is intentionally scoped so that it can be demonstrated end to end. A user can create a job, upload resumes, review rankings, shortlist candidates, and download a PDF report within one session. This makes the implementation practical for presentation and also proves that the individual modules are connected properly.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Functional Requirements

Functional requirements define what ResumeIQ must do for the recruiter. The recruiter should be able to register, log in, create job requirements, view saved jobs, expand job details, delete jobs, upload multiple resumes, view ranked candidates, filter results, shortlist candidates, reject candidates, and download shortlisted candidates as a PDF report. These functions complete the recruiter workflow from job creation to candidate decision.

The backend must parse every supported resume, extract raw text, identify contact information, detect skills, extract common sections, preprocess text, generate semantic relevance scores, calculate skill overlap, calculate resume strength, and store results in SQLite. The dashboard must read from the database and present updated analytics whenever resumes are uploaded or candidate statuses change.

The system must also handle practical edge cases. Unsupported file types should be skipped with a message. Duplicate filenames should not overwrite existing uploaded files. Jobs and candidates should belong to the logged-in recruiter. When a job is deleted, its related candidate and resume records should also be removed.

These requirements make the project more complete than a simple similarity script. ResumeIQ is a web application with login, job management, file handling, analysis, analytics, status decisions, and reporting.

The requirements were selected to balance usefulness and simplicity. Authentication, job management, upload handling, parsing, matching, ranking, analytics, and report export are enough to represent a real recruiter workflow. Features such as candidate portals and interview scheduling were excluded because they would shift the focus away from semantic resume screening.

A major requirement is that analytics must come only from uploaded resumes. This rule prevents misleading dashboard values and makes the system easier to defend during evaluation. If six resumes are uploaded, every chart and summary must reflect those six candidates for the selected job only.

Another requirement is beginner-friendly implementation. For that reason, SQLite is used instead of a server database, Flask templates are used instead of a complex frontend framework, and modules are separated by responsibility. This helps the project remain readable while still demonstrating meaningful AI functionality.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Requirement	Description
Authentication	Recruiter registration, login, logout, and session-based route protection.
Job Management	Create, view, expand, and delete job requirements.

Requirement	Description
Resume Upload	Upload multiple PDF or DOCX resumes for a selected job.
Analysis	Parse resumes, extract skills, calculate semantic and skill scores.
Recruiter Actions	Filter, shortlist, reject, and download shortlisted candidate report.

2.2 Non-Functional Requirements

ResumeIQ must be lightweight, locally executable, and responsive enough for a batch of demonstration resumes. It should run on a normal laptop without a GPU. The application should avoid unnecessary services, containers, or cloud dependencies, because the goal is an understandable academic project rather than a large production platform.

The user interface should be modern and recruiter-readable. Metrics should be shown as percentages rather than raw decimal values. Charts should have clear axis labels and meaningful ranges. The candidate table should be wide enough to compare match, semantic, skill, experience, and strength scores without visual clutter.

Maintainability is also a non-functional requirement. The project should use simple modules and readable functions. Flask routes should coordinate workflow, while separate modules handle parsing, preprocessing, skill extraction, semantic matching, ranking, analytics, and database access. This helps future developers understand where each responsibility belongs.

Reliability is supported through tests and through fallback behavior in preprocessing. The project also keeps uploaded resume analytics tied to the selected job, so one job’s candidates do not pollute another job’s statistics.

The requirements were selected to balance usefulness and simplicity. Authentication, job management, upload handling, parsing, matching, ranking, analytics, and report export are enough to represent a real recruiter workflow. Features such as candidate portals and interview scheduling were excluded because they would shift the focus away from semantic resume screening.

A major requirement is that analytics must come only from uploaded resumes. This rule prevents misleading dashboard values and makes the system easier to defend during evaluation. If six resumes are uploaded, every chart and summary must reflect those six candidates for the selected job only.

Another requirement is beginner-friendly implementation. For that reason, SQLite is used instead of a server database, Flask templates are used instead of a complex frontend framework, and modules are separated by responsibility. This helps the project remain readable while still demonstrating meaningful AI functionality.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

2.3 Software, Hardware, and Feasibility

The software stack consists of Python, Flask, SQLite, pdfplumber, python-docx, NLTK, Sentence Transformers, NumPy, pandas, scikit-learn concepts, TailwindCSS, Chart.js, and ReportLab. These tools were selected because they are widely used, open-source, and suitable for local development.

Hardware requirements are modest. A dual-core processor can run the application, but 8 GB RAM is recommended for a smoother experience when using sentence embeddings. Disk space is required for the virtual environment, Python packages, model cache, generated resumes, uploaded files, and SQLite database. Internet access is needed during first setup to install dependencies and download model resources.

Technical feasibility is high because the project uses mature libraries. Operational feasibility is also high because recruiters do not need to understand the AI internals to use the system. Economic feasibility is strong because the application uses free and open-source tools with no paid APIs or hosted services.

After the model and dependencies are cached, the backend matching pipeline can operate locally. This makes ResumeIQ suitable for college demonstrations where internet access may not be reliable throughout the presentation.

The requirements were selected to balance usefulness and simplicity. Authentication, job management, upload handling, parsing, matching, ranking, analytics, and report export are enough to represent a real recruiter workflow. Features such as candidate portals and interview scheduling were excluded because they would shift the focus away from semantic resume screening.

A major requirement is that analytics must come only from uploaded resumes. This rule prevents misleading dashboard values and makes the system easier to defend during evaluation. If six resumes are uploaded, every chart and summary must reflect those six candidates for the selected job only.

Another requirement is beginner-friendly implementation. For that reason, SQLite is used instead of a server database, Flask templates are used instead of a complex frontend framework, and modules are separated by responsibility. This helps the project remain readable while still demonstrating meaningful AI functionality.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Area	Requirement
Software	Python, Flask, SQLite, NLTK, Sentence Transformers, pdfplumber, python-docx, Chart.js.
Hardware	Dual-core CPU, 8 GB RAM recommended, local storage for model and resumes.
Feasibility	High technical, operational, and economic feasibility for academic use.

CHAPTER 3: SYSTEM DESIGN

3.1 High Level Architecture

ResumeIQ follows a simple monolithic Flask architecture. The browser sends requests to Flask routes in app.py. These routes validate user actions and call helper modules. Utility modules parse resumes, preprocess text, extract skills, calculate resume strength, and access SQLite. AI modules generate embeddings, calculate similarity, rank candidates, and aggregate analytics.

This architecture is intentionally straightforward. It avoids microservices, message queues, or complex deployment layers because the project is meant to be lightweight and academically explainable. A user action can be traced clearly from the dashboard to a route, from the route to a processing function, and from the function to a database record.

The dashboard receives candidate records, metric cards, and chart data from Flask. Chart data is converted into JSON and rendered by Chart.js. Uploaded files are stored in the local uploads folder, while structured results are stored in database/database.db.

This design is useful for evaluation because every layer has a visible role. The user interface collects recruiter input, Flask coordinates requests, modules perform analysis, SQLite persists results, and the dashboard visualizes the final output.

The system design follows a direct request-processing model. The browser submits recruiter input, Flask validates and routes the request, utility modules process resume files, AI modules calculate scores, and SQLite stores the result. This design avoids hidden layers and makes it easy to identify where each output originates.

The separation between resumes and candidates is especially important. A resume record stores parsed document information, while a candidate record stores analysis for a particular job. This prevents the design from mixing raw file data with job-specific scoring data and supports future extension where one resume could be analyzed against multiple jobs.

The design also supports deletion and cleanup. When a job is removed, related candidates and resume analysis records are removed, and uploaded files can be deleted. This prevents old records from affecting new analytics and keeps the local workspace manageable during repeated testing.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Fig. 3.1 High Level System Architecture

Recruiter Browser -> Flask Routes -> Parser and NLP Utilities -> Semantic AI Modules -> SQLite Storage -> Analytics Dashboard

3.2 Workflow, Data Flow, and Use Cases

The recruiter workflow begins with authentication. After logging in, the recruiter creates a job requirement with a title, job description, and required skills. The recruiter then uploads multiple resumes for that job. Each resume is saved, parsed, cleaned, analyzed, scored, and stored. The recruiter returns to the dashboard to see ranked candidates and analytics.

The data flow starts with two major inputs: job requirement and resume files. The job requirement provides the target meaning and required skill list. Resume files provide candidate information. The system extracts text from each resume, detects skills, generates embeddings, compares the resume against the job description, and stores the calculated scores.

The primary actor is the recruiter/admin. The recruiter can register, log in, create jobs, view job details, delete jobs, upload resumes, review candidates, filter candidates, shortlist candidates, reject candidates, and download shortlisted reports. There is deliberately no candidate actor in the system because candidate portal features are outside the scope.

The output is not only a ranked table. ResumeIQ also generates analytics cards, chart distributions, skill frequencies, missing skill counts, domain distribution, status distribution, AI insights, and a shortlisted candidate PDF report.

The system design follows a direct request-processing model. The browser submits recruiter input, Flask validates and routes the request, utility modules process resume files, AI modules calculate scores, and SQLite stores the result. This design avoids hidden layers and makes it easy to identify where each output originates.

The separation between resumes and candidates is especially important. A resume record stores parsed document information, while a candidate record stores analysis for a particular job. This prevents the design from mixing raw file data with job-specific scoring data and supports future extension where one resume could be analyzed against multiple jobs.

The design also supports deletion and cleanup. When a job is removed, related candidates and resume analysis records are removed, and uploaded files can be deleted. This prevents old records from affecting new analytics and keeps the local workspace manageable during repeated testing.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Fig. 3.2 Recruiter Workflow
Login/Register -> Create Job -> Upload Resumes -> Parse and Analyze -> Rank Candidates -> Review Analytics -> Shortlist/Reject -> Download PDF

Fig. 3.3 Data Flow Diagram
Job Description + Required Skills + Resume Files -> Parsing -> Preprocessing -> Skill Extraction -> Embeddings -> Similarity -> Ranking -> Analytics

3.3 Database Design

SQLite is used because it is lightweight, file-based, and easy to explain. The database contains four main tables: users, jobs, resumes, and candidates. The users table stores recruiter account records. The jobs table stores job requirements. The resumes table stores uploaded resume metadata and extracted text. The candidates table stores scoring and recruiter decision data.

The resumes and candidates tables are separated because a resume record stores document-level data, while a candidate record stores job-analysis data. This separation makes the database more organized. The candidate table includes semantic_score, skill_score, experience_score, project_score, resume_strength, final_score, overlap_skills, missing_skills, and status.

The database design supports analytics because every chart can be generated by aggregating stored candidate and resume rows for the selected job. This is why the dashboard can avoid fake or random data. It also makes testing easier because temporary test databases can be created and inspected independently.

The schema is intentionally small. It covers the complete application without requiring an ORM or external server. This supports the project goal of being beginner-friendly and suitable for viva explanation.

The system design follows a direct request-processing model. The browser submits recruiter input, Flask validates and routes the request, utility modules process resume files, AI modules calculate scores, and SQLite stores the result. This design avoids hidden layers and makes it easy to identify where each output originates.

The separation between resumes and candidates is especially important. A resume record stores parsed document information, while a candidate record stores analysis for a particular job. This prevents the design from mixing raw file data with job-specific scoring data and supports future extension where one resume could be analyzed against multiple jobs.

The design also supports deletion and cleanup. When a job is removed, related candidates and resume analysis records are removed, and uploaded files can be deleted. This prevents old records from affecting new analytics and keeps the local workspace manageable during repeated testing.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Table	Purpose
users	Stores recruiter credentials and account creation time.
jobs	Stores job title, description, required skills, and owner.
resumes	Stores filename, raw text, cleaned text, contact details, skills, and sections.
candidates	Stores candidate name, scores, skill gaps, status, and analysis timestamp.

3.4 Security and Access Control Design

ResumeIQ uses a simple recruiter authentication model that is appropriate for a local academic project. A recruiter registers with a username and password, and the password is stored as a hash using Werkzeug security utilities. The application does not store plain-text passwords, which is a basic but important security requirement.

Protected routes use a login_required decorator. This means a user cannot directly access the dashboard, upload route, candidate status route, job deletion route, or shortlisted report route without a valid session. The current user id stored in the session is also used to check whether a job belongs to the logged-in recruiter.

File upload handling is another part of security. ResumeIQ accepts only PDF and DOCX extensions, uses secure_filename before saving a file, and creates a unique upload path to avoid overwriting existing files. These practices reduce accidental file conflicts and unsafe filenames.

The system is not designed as an enterprise security product, but it follows sensible local-app practices. Authentication, route protection, ownership checks, file extension validation, and local-only execution together make the prototype safer and easier to explain.

The system design follows a direct request-processing model. The browser submits recruiter input, Flask validates and routes the request, utility modules process resume files, AI modules calculate scores, and SQLite stores the result. This design avoids hidden layers and makes it easy to identify where each output originates.

The separation between resumes and candidates is especially important. A resume record stores parsed document information, while a candidate record stores analysis for a particular job. This prevents the design from mixing raw file data with job-specific scoring data and supports future extension where one resume could be analyzed against multiple jobs.

The design also supports deletion and cleanup. When a job is removed, related candidates and resume analysis records are removed, and uploaded files can be deleted. This prevents old records from affecting new analytics and keeps the local workspace manageable during repeated testing.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

3.5 Dashboard Information Design

The dashboard is designed around recruiter questions instead of technical model internals. A recruiter first wants to know how many resumes were uploaded, which candidate is strongest, whether the applicant pool is generally relevant, which skills are common, and which required skills are missing. ResumeIQ places these answers near the top of the dashboard through metric cards.

Charts are placed after the cards because they provide broader applicant-pool interpretation. The match score histogram explains candidate quality distribution, the semantic relevance chart explains meaning-level fit, the top skills chart explains available talent, the domain chart explains

role concentration, and the status chart explains recruiter decisions.

The ranked candidate table is full width because candidate comparison is the central task. The table includes score percentages, progress bars, status chips, and actions so that the recruiter can compare candidates without opening a separate page. This layout also improves presentation because the evaluator can see the full ranking logic in one place.

The AI insight panel is placed after the ranked table so that it reads like a summary of the visible evidence. It does not invent new values; it converts analytics into short recruiter-friendly observations such as the most common domain, most missing skill, percentage of resumes above a threshold, and average semantic relevance.

The system design follows a direct request-processing model. The browser submits recruiter input, Flask validates and routes the request, utility modules process resume files, AI modules calculate scores, and SQLite stores the result. This design avoids hidden layers and makes it easy to identify where each output originates.

The separation between resumes and candidates is especially important. A resume record stores parsed document information, while a candidate record stores analysis for a particular job. This prevents the design from mixing raw file data with job-specific scoring data and supports future extension where one resume could be analyzed against multiple jobs.

The design also supports deletion and cleanup. When a job is removed, related candidates and resume analysis records are removed, and uploaded files can be deleted. This prevents old records from affecting new analytics and keeps the local workspace manageable during repeated testing.

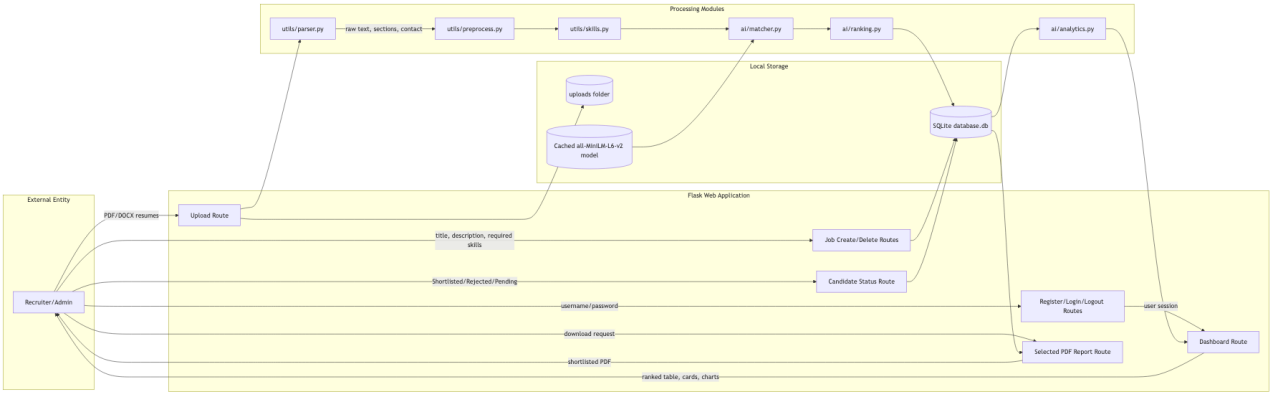
For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

3.6 Workflow Diagram

The workflow diagram explains the complete recruiter journey implemented in ResumeIQ. It begins with registration or login, continues through job creation and multiple resume upload, and ends with candidate ranking, recruiter decision status, and shortlisted PDF download.

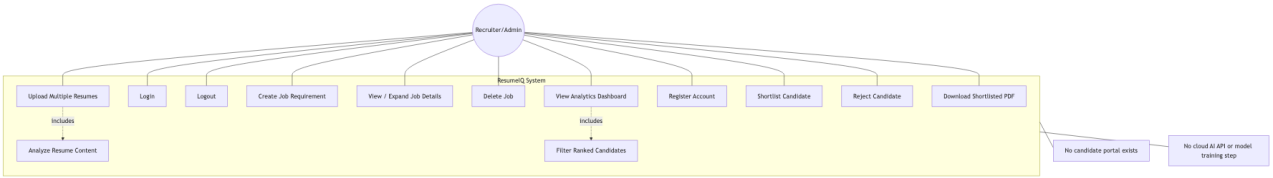
3.7 Data Flow Diagram

The data flow diagram shows how information moves through the Flask routes, processing modules, local model cache, uploads folder, and SQLite database. It also shows that dashboard analytics and selected-candidate reports are generated from stored uploaded-resume records.



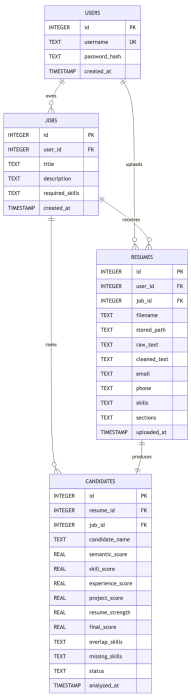
3.8 Use Case Diagram

The use case diagram shows the single recruiter/admin actor and the supported application actions. It also clarifies excluded features such as candidate portal access, cloud AI calls, and model training because those are intentionally outside the project scope.



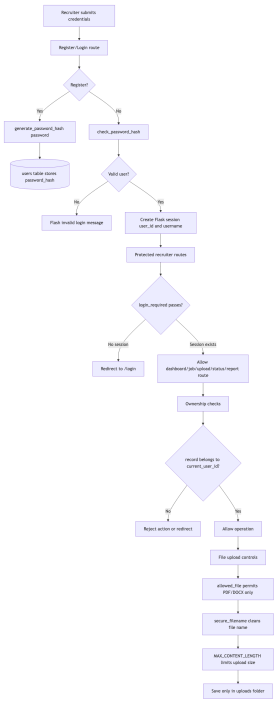
3.9 Database Design

The database design diagram is based on the actual SQLite schema in utils/database.py. It contains users, jobs, resumes, and candidates, with candidate rows storing the job-specific scoring results and recruiter status for each uploaded resume.



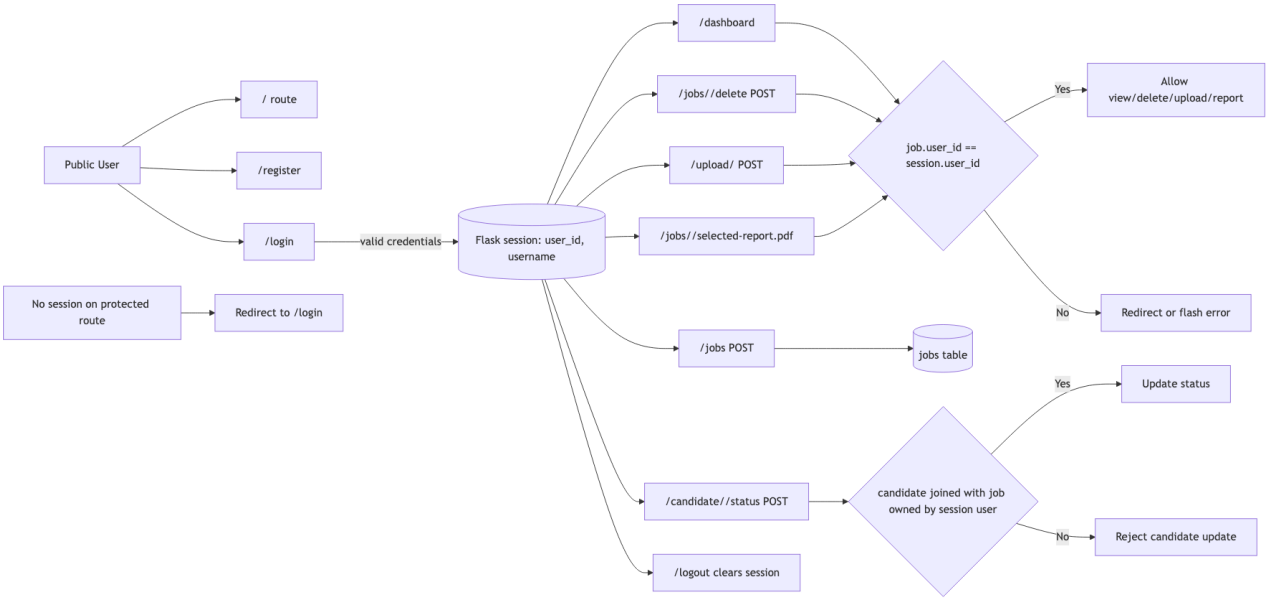
3.10 Security Design

The security design diagram shows the implemented password hashing, session creation, login-required route protection, ownership checks, upload validation, secure filenames, and upload size limit used by the Flask application.



3.11 Access Control Design

The access control diagram separates public routes from authenticated recruiter routes. It also shows how the application checks job ownership and candidate ownership before allowing dashboard access, upload, delete, status update, or shortlisted report download.



CHAPTER 4: SEMANTIC AI AND NLP METHODOLOGY

4.1 Resume Parsing and NLP Preprocessing

ResumeIQ supports PDF and DOCX parsing. PDF resumes are parsed with pdfplumber, which extracts text from each page. DOCX resumes are parsed with python-docx, which reads paragraphs and table cells. The parser returns raw text, email, phone, detected skills, and extracted sections.

Email and phone values are extracted using regular expressions. Candidate names are inferred from early resume lines and cleaned to remove section-label noise such as contact or profile when it appears near the name. This is important for visual resumes where layout text may appear on the same line.

Text preprocessing improves consistency before semantic comparison. The preprocessing pipeline lowercases text, normalizes whitespace, removes punctuation, removes stopwords, tokenizes words, and applies lemmatization when NLTK resources are available. The cleaned text reduces noise and provides a more stable representation of resume content.

The module includes fallback behavior for offline use. If NLTK resources are not available, fallback stopwords and regex tokenization allow the application to continue working. This is useful for presentation environments where internet access may not be reliable after setup.

The NLP pipeline begins with extraction because semantic matching is only as good as the text available to the model. By supporting both PDF and DOCX formats, ResumeIQ covers the most common resume document types. Email, phone, skills, and section extraction add structured information to the otherwise unstructured resume text.

Skill extraction and semantic matching solve different parts of the problem. Skill extraction gives exact evidence that a required technology or tool appears in the resume. Semantic matching captures broader meaning, such as recognizing that building REST services is related to backend API development. The weighted formula combines both perspectives.

The use of all-MiniLM-L6-v2 is suitable because it is small and efficient compared with larger transformer models. It provides enough semantic understanding for a local academic application while keeping execution practical on a normal laptop CPU. This choice supports the requirement that the project remain lightweight.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

4.2 Skill Extraction and Skill Matching

Skill extraction uses transparent dictionaries and regex matching. Skills are grouped into programming languages, frameworks, databases, cloud tools, developer tools, AI and data tools, design skills, marketing skills, business skills, HR and operations skills, finance and sales skills, support and QA skills, and security/infrastructure skills.

The system normalizes common aliases. For example, React Js becomes react, Next Js becomes next.js, Express Js becomes express, Node Js becomes nodejs, and postgres becomes postgresql. This helps avoid false missing skills when the recruiter and resume use different spelling for the same technology.

Skill matching compares detected resume skills with required skills for the job. It produces overlap skills, missing skills, and a skill_score. These values are shown in the ranked table and used in analytics such as most missing skill and average skill match.

This approach is transparent. During a viva, the skill dictionary and alias mapping can be shown directly. The evaluator can understand how a skill was detected and why a missing skill appeared in the dashboard.

The NLP pipeline begins with extraction because semantic matching is only as good as the text available to the model. By supporting both PDF and DOCX formats, ResumeIQ covers the most common resume document types. Email, phone, skills, and section extraction add structured information to the otherwise unstructured resume text.

Skill extraction and semantic matching solve different parts of the problem. Skill extraction gives exact evidence that a required technology or tool appears in the resume. Semantic matching captures broader meaning, such as recognizing that building REST services is related to backend API development. The weighted formula combines both perspectives.

The use of all-MiniLM-L6-v2 is suitable because it is small and efficient compared with larger transformer models. It provides enough semantic understanding for a local academic application while keeping execution practical on a normal laptop CPU. This choice supports the requirement that the project remain lightweight.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

4.3 Semantic Embeddings and Ranking

ResumeIQ uses Sentence Transformers with the all-MiniLM-L6-v2 model to generate dense semantic embeddings. A semantic embedding is a vector representation of text meaning. The resume text and job description are converted into vectors, and cosine similarity is used to measure how close they are.

The same method is applied to the experience section and project section. This allows ResumeIQ to calculate overall semantic score, experience relevance, and project relevance. These separate values help recruiters understand whether a candidate is generally relevant, whether their work history is relevant, and whether their project experience is relevant.

The final ranking formula combines semantic similarity, skill match, experience relevance, and project relevance. Semantic similarity has a weight of 40 percent, skill match has 35 percent, experience relevance has 15 percent, and project relevance has 10 percent. This formula balances meaning-level fit with explicit required skill coverage.

Resume completeness is calculated separately using rule-based indicators such as resume length, number of detected skills, presence of experience, projects, education, certifications, email, and phone number. This score helps identify whether a resume is well structured, but it does not replace the job match score.

The NLP pipeline begins with extraction because semantic matching is only as good as the text available to the model. By supporting both PDF and DOCX formats, ResumeIQ covers the most common resume document types. Email, phone, skills, and section extraction add structured information to the otherwise unstructured resume text.

Skill extraction and semantic matching solve different parts of the problem. Skill extraction gives exact evidence that a required technology or tool appears in the resume. Semantic matching captures broader meaning, such as recognizing that building REST services is related to backend API development. The weighted formula combines both perspectives.

The use of all-MiniLM-L6-v2 is suitable because it is small and efficient compared with larger transformer models. It provides enough semantic understanding for a local academic application while keeping execution practical on a normal laptop CPU. This choice supports the requirement that the project remain lightweight.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Fig. 4.1 Semantic Matching Pipeline

Resume Text -> Embedding
Job Description -> Embedding
Cosine Similarity -> Semantic Score
Experience Section -> Experience Relevance
Project Section -> Project Relevance

Component	Weight / Role
Semantic Similarity	40% of final score
Skill Match	35% of final score
Experience Relevance	15% of final score
Project Relevance	10% of final score
Resume Completeness	Displayed quality signal, not part of final formula

4.6 Example Semantic Matching Behavior

A useful way to understand ResumeIQ is to compare exact keyword matching with semantic matching. If a job asks for backend API development, a resume sentence such as built REST APIs using FastAPI should be considered relevant even though the wording is not identical. Sentence embeddings help capture this relationship.

Similarly, a data science role may mention predictive insights, dashboards, business data, and machine learning. A resume that discusses pandas, SQL, exploratory analysis, forecasting, and reporting may still be semantically relevant even if every required phrase is not repeated exactly. This is why semantic similarity is valuable.

At the same time, semantic similarity alone is not enough. A candidate might write broadly about software systems and receive some semantic relevance, but still lack required skills such as Docker, PostgreSQL, or FastAPI. ResumeIQ therefore combines semantic relevance with skill overlap to produce a more balanced final score.

This example behavior is demonstrated in the included presentation cases. Backend resumes rank higher for backend jobs, data resumes rank higher for data jobs, and DevOps/cloud resumes rank higher for infrastructure jobs. The system therefore demonstrates both semantic understanding and explicit skill-gap detection.

The NLP pipeline begins with extraction because semantic matching is only as good as the text available to the model. By supporting both PDF and DOCX formats, ResumeIQ covers the most common resume document types. Email, phone, skills, and section extraction add structured information to the otherwise unstructured resume text.

Skill extraction and semantic matching solve different parts of the problem. Skill extraction gives exact evidence that a required technology or tool appears in the resume. Semantic matching captures broader meaning, such as recognizing that building REST services is related to backend API development. The weighted formula combines both perspectives.

The use of all-MiniLM-L6-v2 is suitable because it is small and efficient compared with larger transformer models. It provides enough semantic understanding for a local academic application while keeping execution practical on a normal laptop CPU. This choice supports the requirement that the project remain lightweight.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

4.7 Score Interpretation for Recruiters

ResumeIQ converts raw similarity values into percentages because recruiters should not need to interpret decimal embedding scores. A semantic similarity of 0.62 is easier to understand when shown as 62 percent semantic relevance. This improves readability while preserving the mathematical meaning of the underlying comparison.

The final match percentage is a weighted decision-support score. It is not a guarantee of hiring suitability. A high score means that the resume text, detected skills, experience section, and project section align well with the job requirement. A low score means that the uploaded resume does not provide enough matching evidence for that particular role.

Skill match percentage is interpreted differently from semantic relevance. Skill match is based on required-skill coverage, so it directly answers whether the candidate mentions the technologies requested by the recruiter. Semantic relevance is broader and can still be useful when a candidate uses different wording or describes related work.

Experience and project relevance provide extra explanation for why a candidate ranks high or low. A candidate may have many skills listed but weak experience relevance, or a candidate may have strong project relevance but miss one required tool. Showing these separate metrics helps the recruiter understand the composition of the final score.

The NLP pipeline begins with extraction because semantic matching is only as good as the text available to the model. By supporting both PDF and DOCX formats, ResumeIQ covers the most common resume document types. Email, phone, skills, and section extraction add structured information to the otherwise unstructured resume text.

Skill extraction and semantic matching solve different parts of the problem. Skill extraction gives exact evidence that a required technology or tool appears in the resume. Semantic matching captures broader meaning, such as recognizing that building REST services is related to backend API development. The weighted formula combines both perspectives.

The use of all-MiniLM-L6-v2 is suitable because it is small and efficient compared with larger transformer models. It provides enough semantic understanding for a local academic application while keeping execution practical on a normal laptop CPU. This choice supports the requirement that the project remain lightweight.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic

similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

CHAPTER 5: IMPLEMENTATION

5.1 Flask Application and Routes

The Flask application is implemented in app.py. It contains routes for home redirection, recruiter registration, login, logout, dashboard rendering, job creation, job deletion, resume upload, candidate status updates, and shortlisted candidate PDF download. The route functions coordinate user requests and call processing modules as needed.

Authentication uses session storage and Werkzeug password hashing. The login_required decorator protects recruiter-only routes. Job and candidate actions check that records belong to the logged-in recruiter. This keeps the application simple but still structured enough for local use.

The upload route validates file extensions, secures filenames, saves each file in the uploads folder, and calls the analysis function. The analysis function parses the resume, preprocesses text, extracts skills, calculates scores, and inserts records into SQLite.

Job deletion removes related candidate and resume records and deletes local uploaded files. This keeps the project clean and prevents old job data from affecting new job analytics.

The implementation follows the same structure as the design. The Flask route does not attempt to perform every task directly. Instead, it coordinates smaller functions that parse files, clean text, extract skills, calculate relevance, store records, and prepare dashboard data. This keeps the code easier to inspect and maintain.

The dashboard implementation is not only a visual layer; it is part of the recruiter workflow. The scrollable jobs panel helps when many jobs exist, the details expander shows job description and required skills, the full-width candidate table improves comparison, and the PDF export turns shortlist decisions into a shareable document.

ReportLab is used for the shortlisted report because it can generate PDFs directly from Python. This avoids depending on external office conversion tools and makes the export feature part of the application itself. The PDF contains only candidates actually marked as shortlisted for the selected job.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Route	Purpose
/	Redirects to login or dashboard.
/register	Creates recruiter account.
/login	Authenticates recruiter.

Route	Purpose
/dashboard	Displays jobs, analytics, and ranked candidates.
/jobs	Creates job requirement.
/upload/	Uploads and analyzes resumes.
/candidate//status	Updates shortlist/reject status.
/jobs//selected-report.pdf	Downloads shortlisted candidate report.

5.2 AI, Utility, and Dashboard Modules

The ai folder contains embeddings.py, matcher.py, ranking.py, and analytics.py. embeddings.py loads the all-MiniLM-L6-v2 model and generates vectors. matcher.py calculates cosine similarity and section relevance. ranking.py applies the weighted scoring formula. analytics.py aggregates recruiter metrics and prepares Chart.js data.

The utils folder contains parser.py, preprocess.py, skills.py, scoring.py, and database.py. parser.py handles PDF and DOCX extraction. preprocess.py applies NLTK-based cleaning. skills.py detects skills and compares overlap. scoring.py calculates resume strength. database.py provides SQLite connection, initialization, fetch, and execute helpers.

The dashboard is designed for recruiter readability. The top section contains job creation, a scrollable job list with expandable details, and resume upload. The analytics section contains metric cards and charts. The ranked candidate table occupies the full page width so that match, semantic, skill, experience, resume strength, status, and actions can be compared comfortably.

Chart.js renders match score distribution, semantic relevance distribution, top detected skills, domain distribution, and shortlisted versus rejected status. The AI insight panel converts metrics into recruiter-friendly statements generated only from uploaded resume data.

The implementation follows the same structure as the design. The Flask route does not attempt to perform every task directly. Instead, it coordinates smaller functions that parse files, clean text, extract skills, calculate relevance, store records, and prepare dashboard data. This keeps the code easier to inspect and maintain.

The dashboard implementation is not only a visual layer; it is part of the recruiter workflow. The scrollable jobs panel helps when many jobs exist, the details expander shows job description and required skills, the full-width candidate table improves comparison, and the PDF export turns shortlist decisions into a shareable document.

ReportLab is used for the shortlisted report because it can generate PDFs directly from Python. This avoids depending on external office conversion tools and makes the export feature part of the application itself. The PDF contains only candidates actually marked as shortlisted for the selected job.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a

measurable effect on the final recruiter dashboard.

Fig. 5.1 Module Organization
app.py: Flask routes and workflow coordination
ai/: embeddings, matching, ranking, analytics
utils/: parser, preprocessing, skills, scoring, database
templates/: login, register, dashboard
static/js/dashboard.js: Chart.js rendering

5.3 Shortlisted Candidate PDF Export

ResumeIQ includes a PDF export route for shortlisted candidates. After the recruiter marks candidates as Shortlisted, the download button generates a report containing rank, candidate name, match percentage, semantic percentage, skill match percentage, and email. ReportLab is used to produce this PDF dynamically.

This feature is useful because recruiters often need to share a final list outside the screening dashboard. Instead of copying table values manually, the system produces a clean report for the selected job. If no candidates are shortlisted, the PDF states that no shortlisted candidates exist.

The export feature also demonstrates that the application supports action beyond analytics. It takes recruiter decisions stored in the database and converts them into a formal output document.

The exported report is intentionally concise. It does not attempt to repeat the entire dashboard. Instead, it records the recruiter decision outcome for a job and gives the essential comparison fields needed for follow-up review.

The implementation follows the same structure as the design. The Flask route does not attempt to perform every task directly. Instead, it coordinates smaller functions that parse files, clean text, extract skills, calculate relevance, store records, and prepare dashboard data. This keeps the code easier to inspect and maintain.

The dashboard implementation is not only a visual layer; it is part of the recruiter workflow. The scrollable jobs panel helps when many jobs exist, the details expander shows job description and required skills, the full-width candidate table improves comparison, and the PDF export turns shortlist decisions into a shareable document.

ReportLab is used for the shortlisted report because it can generate PDFs directly from Python. This avoids depending on external office conversion tools and makes the export feature part of the application itself. The PDF contains only candidates actually marked as shortlisted for the selected job.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

5.4 End-to-End Analysis Workflow

The analysis workflow begins when the recruiter uploads one or more resumes for a selected job. For each file, the application saves the document, extracts raw text, identifies contact information, detects sections, cleans the text, extracts skills, and compares the resume against the job requirement.

After extraction, the application generates embeddings for the job description, full resume text, experience section, and project section. Cosine similarity is then calculated to estimate semantic relevance. Skill overlap is calculated separately by comparing normalized required skills with normalized detected skills.

The ranking module combines semantic similarity, skill match, experience relevance, and project relevance. The resulting score is stored as a candidate record connected to the job. This record also stores supporting values so the dashboard can show not only the final match percentage but also the reason behind it.

Finally, the analytics module reads all candidates for the active job and calculates cards, charts, and insight text. This makes the workflow fully connected: uploaded files create records, records create rankings, rankings create analytics, and recruiter actions create shortlisted or rejected status counts.

The implementation follows the same structure as the design. The Flask route does not attempt to perform every task directly. Instead, it coordinates smaller functions that parse files, clean text, extract skills, calculate relevance, store records, and prepare dashboard data. This keeps the code easier to inspect and maintain.

The dashboard implementation is not only a visual layer; it is part of the recruiter workflow. The scrollable jobs panel helps when many jobs exist, the details expander shows job description and required skills, the full-width candidate table improves comparison, and the PDF export turns shortlist decisions into a shareable document.

ReportLab is used for the shortlisted report because it can generate PDFs directly from Python. This avoids depending on external office conversion tools and makes the export feature part of the application itself. The PDF contains only candidates actually marked as shortlisted for the selected job.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

5.5 Error Handling and Local Execution

ResumeIQ includes basic error handling for common local-application problems. If a recruiter uploads an unsupported file type, the application avoids processing it. If a resume contains little extractable text, the parser still returns a safe empty or partial result instead of stopping the entire upload batch.

The local model loading process is also handled carefully. Sentence Transformers may take time to load during the first run, but after the model is available, the same model object can be reused for scoring. This keeps the application practical for repeated uploads during presentation.

Database operations are kept simple and explicit. Tables are created during initialization, and helper functions open SQLite connections for inserts, updates, deletes, and queries. This approach avoids heavy database frameworks and makes it easier for students to explain how data moves through the system.

The project is optimized for local execution on a laptop. It does not require Docker, Kubernetes, cloud storage, external APIs, or a separate model-training service. This keeps deployment simple and allows the evaluator to run the same application from the project folder.

The implementation follows the same structure as the design. The Flask route does not attempt to perform every task directly. Instead, it coordinates smaller functions that parse files, clean text, extract skills, calculate relevance, store records, and prepare dashboard data. This keeps the code easier to inspect and maintain.

The dashboard implementation is not only a visual layer; it is part of the recruiter workflow. The scrollable jobs panel helps when many jobs exist, the details expander shows job description and required skills, the full-width candidate table improves comparison, and the PDF export turns shortlist decisions into a shareable document.

ReportLab is used for the shortlisted report because it can generate PDFs directly from Python. This avoids depending on external office conversion tools and makes the export feature part of the application itself. The PDF contains only candidates actually marked as shortlisted for the selected job.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

CHAPTER 6: TESTING AND VALIDATION

6.1 Automated Testing

ResumeIQ includes automated tests for core modules. Unit tests cover parsing, preprocessing, skill extraction, scoring, analytics aggregation, and PDF report generation. These tests ensure that individual parts of the system behave correctly even before the full application flow is tested.

Integration tests use Flask test clients to simulate recruiter actions such as registering, logging in, creating jobs, uploading resumes, rendering the dashboard, and downloading PDF reports. This confirms that the application routes, database functions, and analysis modules work together.

Skill alias tests are especially important. Recruiters may enter React Js, Next Js, Express Js, or Node Js, while resumes may use slightly different spellings. The tests ensure these aliases normalize correctly and produce expected skill overlap.

The tests also protect the analytics layer. Since every chart and metric must come from uploaded resumes, tests verify that dashboard outputs are based on database rows and that chart totals match candidate counts.

Testing is necessary because ResumeIQ depends on a chain of processing steps. A resume must be saved correctly, parsed correctly, cleaned correctly, analyzed correctly, stored correctly, and displayed correctly. If any one stage fails, the recruiter sees incorrect ranking or analytics. This is why both module and pipeline tests are included.

The 100-resume pipeline check is especially useful because it validates the generated visual PDF resumes with the actual parser and scoring flow. It confirms that the demonstration dataset is not just present in the folder but can actually be uploaded, analyzed, ranked, and counted by the dashboard.

Offline testing is also important for presentation reliability. Once the required packages and model are cached, the backend should produce the same summary without needing internet access. This protects the demonstration from network problems and proves that semantic matching is local after setup.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Testing Area	Validation Purpose
Module tests	Verify parser, preprocessing, skills, scoring, analytics, and PDF route.
Integration tests	Verify Flask workflow from login to dashboard.
Full pipeline test	Upload and analyze all 100 generated PDFs.
Offline comparison	Check cached execution consistency.

6.2 Full Pipeline and Manual Validation

The full pipeline check creates a temporary database, registers a recruiter, creates a realistic backend job, uploads all 100 generated visual PDF resumes, parses them, scores them, creates candidate records, generates dashboard metrics, and verifies chart totals. This validates the entire screening workflow.

The full check confirms that 100 resumes are uploaded and 100 candidates are created. It also verifies that distribution totals match the number of candidates, which proves that analytics are aligned with real uploaded data. Offline and cached execution checks confirm that summaries remain consistent after the required resources have been downloaded.

Manual testing is supported through the PRESENTATION_INPUTS.md guide. It lists job descriptions, required skills, PDF upload sets, and expected top candidates. The recommended cases are Backend API Hiring, Data Science Analytics Hiring, and DevOps Cloud Hiring because they clearly show relevant candidates ranking above unrelated resumes.

For the Backend API case, Aarav Sharma should rank first. For the Data Science case, Sia Bakshi should rank first with Rudra Talwar also high. For the DevOps Cloud case, Sneha Sarin and Kartik Patil should be close near the top. Slight score variation is acceptable if the job description wording changes.

Testing is necessary because ResumeIQ depends on a chain of processing steps. A resume must be saved correctly, parsed correctly, cleaned correctly, analyzed correctly, stored correctly, and displayed correctly. If any one stage fails, the recruiter sees incorrect ranking or analytics. This is why both module and pipeline tests are included.

The 100-resume pipeline check is especially useful because it validates the generated visual PDF resumes with the actual parser and scoring flow. It confirms that the demonstration dataset is not just present in the folder but can actually be uploaded, analyzed, ranked, and counted by the dashboard.

Offline testing is also important for presentation reliability. Once the required packages and model are cached, the backend should produce the same summary without needing internet access. This protects the demonstration from network problems and proves that semantic matching is local after setup.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

6.3 Test Case Design and Expected Outcomes

The manual test cases are designed to show both positive and negative ranking behavior. Each case includes six resumes: some are strongly related to the job, some are partially related, and some are intentionally unrelated. This mixture is useful because it creates visible ranking separation and meaningful analytics.

For example, the Backend API Hiring case includes a backend Python developer, a full stack engineer, DevOps and cloud profiles, a data scientist, and a graphic designer. The expected behavior is that the backend candidate ranks first while unrelated profiles receive lower scores.

The Data Science Analytics case includes data analyst, data scientist, machine learning, NLP, finance, and graphic design profiles. The expected behavior is that data-oriented resumes rank above unrelated design profiles. The DevOps Cloud case similarly checks whether cloud and DevOps profiles rise above unrelated business or sales profiles.

These cases are not meant to prove exact fixed percentages. Semantic scores can move slightly if the job wording changes. The important expected outcome is ranking order, skill overlap, missing skills, and chart totals that match the uploaded resume count.

Testing is necessary because ResumeIQ depends on a chain of processing steps. A resume must be saved correctly, parsed correctly, cleaned correctly, analyzed correctly, stored correctly, and displayed correctly. If any one stage fails, the recruiter sees incorrect ranking or analytics. This is why both module and pipeline tests are included.

The 100-resume pipeline check is especially useful because it validates the generated visual PDF resumes with the actual parser and scoring flow. It confirms that the demonstration dataset is not just present in the folder but can actually be uploaded, analyzed, ranked, and counted by the dashboard.

Offline testing is also important for presentation reliability. Once the required packages and model are cached, the backend should produce the same summary without needing internet access. This protects the demonstration from network problems and proves that semantic matching is local after setup.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

CHAPTER 7: RESULTS AND DISCUSSION

7.1 System Output

The final system produces a working recruiter dashboard. When resumes are uploaded, the system displays ranked candidates with match percentage, semantic percentage, skill match percentage, experience score, resume strength, status, and actions. The recruiter can filter results, shortlist candidates, reject candidates, and download shortlisted candidates.

The analytics cards summarize the active job. They include total resumes, average match, average semantic score, average skill match, shortlisted candidates, top skill, most missing skill, average resume completeness, top candidate, experience relevance, and project relevance. These values help recruiters understand the applicant pool without opening every resume first.

The visual charts support decision-making by showing score distribution, skill frequency, candidate domain distribution, and status distribution. This turns the project from a simple ranking table into a recruiter analytics system.

The dashboard also improves presentation quality because it allows the evaluator to see the effect of uploading different resume combinations. The chart and card values change based on the selected job and uploaded resumes.

The result cases are intentionally mixed with relevant and less relevant resumes. This makes the ranking meaningful. If every uploaded resume belonged to the same domain, the dashboard would not clearly show distribution or contrast. By mixing related and unrelated candidates, the analytics demonstrate how the system separates stronger matches from weaker ones.

The expected scores are approximate because wording in the job description affects semantic similarity. This is normal for semantic systems. The important result is the relative behavior: backend resumes should rank high for backend jobs, data resumes for data jobs, and DevOps/cloud resumes for infrastructure jobs.

The results also show why multiple metrics are useful. A candidate may have high resume completeness but low skill match, or high semantic relevance but missing specific tools. The recruiter can use these differences to decide whether a candidate deserves closer review.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

7.2 Demo Results and Interpretation

In the Backend API Hiring case, the backend Python developer resume ranks first because it strongly overlaps with required backend skills such as Python, Flask, FastAPI, PostgreSQL, SQLite, and Git. The average score is lower than the top score because unrelated resumes are intentionally included to demonstrate distribution.

In the Data Science Analytics case, data analyst, data scientist, machine learning, and NLP resumes rank above unrelated design or finance resumes. This demonstrates semantic matching across related technical domains. In the DevOps Cloud case, cloud and DevOps resumes rank close to the top, showing that related infrastructure roles are recognized by the system.

Scores should be interpreted as decision-support indicators, not final hiring decisions. A candidate with a slightly lower score may still be suitable if their experience is strong or if the job description is broad. ResumeIQ helps recruiters prioritize review, but it does not replace human judgment.

The system improves over basic keyword filtering because semantic embeddings can capture meaning-level similarity. At the same time, explicit skill matching remains important because recruiters often have required technologies. Combining both signals produces a more useful ranking than using only one approach.

The result cases are intentionally mixed with relevant and less relevant resumes. This makes the ranking meaningful. If every uploaded resume belonged to the same domain, the dashboard would not clearly show distribution or contrast. By mixing related and unrelated candidates, the analytics demonstrate how the system separates stronger matches from weaker ones.

The expected scores are approximate because wording in the job description affects semantic similarity. This is normal for semantic systems. The important result is the relative behavior: backend resumes should rank high for backend jobs, data resumes for data jobs, and DevOps/cloud resumes for infrastructure jobs.

The results also show why multiple metrics are useful. A candidate may have high resume completeness but low skill match, or high semantic relevance but missing specific tools. The recruiter can use these differences to decide whether a candidate deserves closer review.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Case	Expected Top Candidate	Main Observation
Backend API Hiring	Aarav Sharma	Backend resume ranks highest.
Data Science Analytics	Sia Bakshi	Data profiles rank above unrelated roles.
DevOps Cloud Hiring	Sneha Sarin / Kartik Patil	Cloud and DevOps profiles compete closely.
Frontend Product UI	Varun Kale	Frontend alias matching works.
Business Product Operations	Rahul Jain	Business role screening is supported.
QA Automation	Vivaan Kohli	Specialized QA resume ranks highest.

7.3 Recruiter Decision Support Discussion

ResumeIQ is best understood as a recruiter decision-support system. The system does not make a final hiring decision; it organizes evidence so the recruiter can review candidates faster. This distinction is important because resumes contain incomplete and self-reported information, and human judgment is still required before interview selection.

The dashboard supports decision-making by showing both individual and group-level information. Individual information appears in the ranked table, where each candidate has match, semantic, skill, experience, and strength scores. Group-level information appears in the cards and charts, where the recruiter can understand the overall applicant pool.

The most useful result is not simply the highest candidate score. The useful result is the combination of candidate order, skill gaps, score distribution, and status tracking. For example, if the top candidate is strong but most applicants are weak, the recruiter may continue sourcing. If many candidates are strong, the recruiter can shortlist more confidently.

This discussion also shows why transparent scoring matters. Because ResumeIQ displays separate score components, a recruiter can understand whether a candidate is ranked high because of exact skills, semantic experience, or project relevance. This is more explainable than a black-box score shown without supporting evidence.

The result cases are intentionally mixed with relevant and less relevant resumes. This makes the ranking meaningful. If every uploaded resume belonged to the same domain, the dashboard would not clearly show distribution or contrast. By mixing related and unrelated candidates, the analytics demonstrate how the system separates stronger matches from weaker ones.

The expected scores are approximate because wording in the job description affects semantic similarity. This is normal for semantic systems. The important result is the relative behavior: backend resumes should rank high for backend jobs, data resumes for data jobs, and DevOps/cloud resumes for infrastructure jobs.

The results also show why multiple metrics are useful. A candidate may have high resume completeness but low skill match, or high semantic relevance but missing specific tools. The recruiter can use these differences to decide whether a candidate deserves closer review.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

CHAPTER 8: USER GUIDE

8.1 Installation and Startup

The project is run locally using Python. The user creates a virtual environment, installs dependencies from requirements.txt, and starts the application with python app.py. The browser opens http://127.0.0.1:5001 to access ResumeIQ. The README contains detailed setup steps for macOS, Linux, and Windows.

The first setup needs internet access for Python packages and model download. After the model is cached, backend matching can run locally. The generated visual resumes are stored in synthetic_visual_resumes_pdf and can be used for demonstration and testing.

If the app is already running, the user can open the local URL directly. If port 5001 is busy, the existing Flask process should be stopped or the port changed in app.py.

During presentation, it is better to prepare the virtual environment and cached model before the evaluation begins. This avoids waiting for package downloads or model initialization during the demonstration.

The user workflow was designed so that a first-time evaluator can understand the app quickly. The recruiter starts from login, creates a job, uploads resumes, and immediately sees ranked output. The same screen provides analytics and actions, so the user does not need to navigate through many pages.

For demonstrations, it is important to use the prepared cases exactly. Creating a fresh job for each case and uploading only the listed PDFs ensures that the output matches the expected explanation. Extra resumes in the same job will change averages, charts, and missing skill counts.

The shortlisted PDF export should be demonstrated after marking one or two candidates as Shortlisted. This shows that ResumeIQ is not only calculating scores but also supporting a recruiter decision process that ends with a usable report.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

8.2 Recruiter Usage Steps

The recruiter first registers or logs in. After login, the dashboard appears. The recruiter creates a job requirement using the job title, job description, and required skills fields. Saved jobs appear in the scrollable jobs panel, and each job has an expandable details section.

After selecting a job, the recruiter uploads PDF or DOCX resumes. ResumeIQ analyzes the files and updates the dashboard. The recruiter reviews metric cards, charts, AI insights, and the ranked candidate table. Filters can be applied by score, skill, and status.

The recruiter can mark candidates as Shortlisted or Rejected. Shortlisted candidates appear in the status chart and can be exported using the Download Shortlisted PDF button. This creates a clean final report for the selected job.

For best results, a fresh job should be created for each demo case. Mixing resumes from multiple cases into one job will change the analytics and may make the expected demonstration results harder to compare.

The user workflow was designed so that a first-time evaluator can understand the app quickly. The recruiter starts from login, creates a job, uploads resumes, and immediately sees ranked output. The same screen provides analytics and actions, so the user does not need to navigate through many pages.

For demonstrations, it is important to use the prepared cases exactly. Creating a fresh job for each case and uploading only the listed PDFs ensures that the output matches the expected explanation. Extra resumes in the same job will change averages, charts, and missing skill counts.

The shortlisted PDF export should be demonstrated after marking one or two candidates as Shortlisted. This shows that ResumeIQ is not only calculating scores but also supporting a recruiter decision process that ends with a usable report.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

Fig. 8.1 Recruiter Dashboard Flow

Create job -> Upload resumes -> Review analytics -> Review ranked table -> Shortlist candidates -> Download selected report

8.3 Demonstration Procedure for Evaluation

For the best presentation, the application should be started before the evaluator begins checking the project. The browser should be opened at http://127.0.0.1:5001, and the recruiter account should be ready. This avoids setup delay and lets the demonstration focus on the actual functionality.

The recommended first demo is Backend API Hiring. Create the job using the description and required skills from PRESENTATION_INPUTS.md, upload the six listed resumes, and explain why Aarav Sharma ranks first. Then show match score distribution, top detected skills, most missing skill, and the ranked candidate table.

The second demo can be Data Science Analytics Hiring, followed by DevOps Cloud Hiring. These three cases show that ResumeIQ works across different domains. They also demonstrate that the same semantic matching pipeline can support backend, analytics, and infrastructure hiring

scenarios.

Finally, shortlist one or two candidates and download the shortlisted PDF report. This final step shows that ResumeIQ supports recruiter decision output, not only analysis. It is a strong closing demonstration because it connects ranking to a practical report.

The user workflow was designed so that a first-time evaluator can understand the app quickly. The recruiter starts from login, creates a job, uploads resumes, and immediately sees ranked output. The same screen provides analytics and actions, so the user does not need to navigate through many pages.

For demonstrations, it is important to use the prepared cases exactly. Creating a fresh job for each case and uploading only the listed PDFs ensures that the output matches the expected explanation. Extra resumes in the same job will change averages, charts, and missing skill counts.

The shortlisted PDF export should be demonstrated after marking one or two candidates as Shortlisted. This shows that ResumeIQ is not only calculating scores but also supporting a recruiter decision process that ends with a usable report.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

CHAPTER 9: LIMITATIONS AND FUTURE SCOPE

9.1 Limitations

ResumeIQ depends on the quality of extracted text. If a PDF is image-only or scanned, pdfplumber may not extract useful text. Such resumes would need OCR support in a future version. The current section extraction also depends on common headings such as experience, projects, education, and certifications, so unusual resume formats may reduce section-level scoring quality.

Skill extraction is dictionary-based. This makes it transparent and explainable, but it also means new technologies and domain terms must be added manually. A resume may contain a valid skill that is not yet in the dictionary. Similarly, domain distribution depends on predefined domain rules.

The system is designed as a local academic project, not a complete enterprise ATS. It does not include candidate accounts, interview scheduling, email automation, resume preview, audit logs, or multi-recruiter permissions. These are reasonable exclusions for the current project scope.

The score should not be interpreted as an automatic hiring decision. ResumeIQ is a decision-support system that helps prioritize candidates, but recruiters should still review the resume content, projects, experience, and job context before making final selections.

The limitations are realistic for a local academic system. Text-based PDFs work well, but scanned image resumes require OCR. Dictionary-based skill extraction is explainable, but it must be maintained as new technologies and domain skills appear. These limitations do not weaken the project; they show a clear understanding of its boundaries.

Future scope can be added gradually without changing the core idea. OCR, richer dictionaries, CSV export, resume preview, recruiter notes, configurable weights, and job templates can be built on top of the current database and dashboard structure. This proves that the architecture is extendable.

The project can also be improved by bundling frontend assets locally. Currently TailwindCSS and Chart.js are loaded through CDN for convenience. Local bundling would make the visual dashboard fully independent of browser internet access after installation.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

9.2 Future Scope

Future improvements can include OCR for scanned resumes, expanded skill dictionaries, recruiter notes, CSV export, resume preview, role templates, interview pipeline tracking, and more advanced filtering. The dashboard can also include historical hiring trends if the system is extended for long-term use.

The UI can be improved further by bundling TailwindCSS and Chart.js locally for fully offline visual operation. The analytics can also be extended with comparison between jobs, candidate source tracking, and score explanation panels for each candidate.

Another possible enhancement is adding configurable scoring weights. Recruiters may want skill match to matter more for technical jobs or semantic relevance to matter more for broad business roles. A future settings panel could allow these weights to be adjusted per job.

The project can also be extended with role-specific templates. A recruiter could select Backend Developer, Data Scientist, DevOps Engineer, or QA Automation Engineer and automatically receive suggested required skills and evaluation weights.

The limitations are realistic for a local academic system. Text-based PDFs work well, but scanned image resumes require OCR. Dictionary-based skill extraction is explainable, but it must be maintained as new technologies and domain skills appear. These limitations do not weaken the project; they show a clear understanding of its boundaries.

Future scope can be added gradually without changing the core idea. OCR, richer dictionaries, CSV export, resume preview, recruiter notes, configurable weights, and job templates can be built on top of the current database and dashboard structure. This proves that the architecture is extendable.

The project can also be improved by bundling frontend assets locally. Currently TailwindCSS and Chart.js are loaded through CDN for convenience. Local bundling would make the visual dashboard fully independent of browser internet access after installation.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic

similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

9.3 Practical Future Enhancements

A practical next improvement is adding OCR support for scanned resumes. Many real-world resumes are exported as text-based PDFs, but some are scanned images. OCR would allow ResumeIQ to extract text from image-only documents and reduce parsing failures.

Another useful improvement is a resume preview panel. Recruiters could click a candidate and see the original resume or extracted text beside the score explanation. This would make the review process faster because the recruiter would not need to manually search for uploaded files.

The analytics could also be extended with saved comparison reports. A recruiter may want to compare two jobs, compare applicant pools, or export charts as part of a hiring summary. These features can be added because ResumeIQ already stores structured job, resume, candidate, and status data.

Finally, the skill dictionary can be made easier to maintain through an admin interface. Instead of editing Python files, a recruiter or administrator could add new skills and aliases through the dashboard. This would make the system more adaptable to new technologies and domains.

The limitations are realistic for a local academic system. Text-based PDFs work well, but scanned image resumes require OCR. Dictionary-based skill extraction is explainable, but it must be maintained as new technologies and domain skills appear. These limitations do not weaken the project; they show a clear understanding of its boundaries.

Future scope can be added gradually without changing the core idea. OCR, richer dictionaries, CSV export, resume preview, recruiter notes, configurable weights, and job templates can be built on top of the current database and dashboard structure. This proves that the architecture is extendable.

The project can also be improved by bundling frontend assets locally. Currently TailwindCSS and Chart.js are loaded through CDN for convenience. Local bundling would make the visual dashboard fully independent of browser internet access after installation.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

CHAPTER 10: CONCLUSION AND REFERENCES

10.1 Conclusion

ResumeIQ successfully demonstrates a practical semantic AI resume screening and candidate ranking system. It combines resume parsing, NLP preprocessing, skill extraction, sentence embeddings, cosine similarity, weighted ranking, and recruiter analytics in one local Flask application. The system supports the complete recruiter workflow from job creation to shortlist PDF export.

The project is stronger than a basic resume ranking tool because it includes semantic comparison, skill gaps, section relevance, resume completeness, dynamic analytics, status decisions, and generated visual resume test cases. It remains lightweight because it uses SQLite, simple Flask templates, and modular Python code instead of unnecessary infrastructure.

Overall, ResumeIQ meets its goal as an academically explainable and locally executable recruiter assistance system. It helps recruiters understand candidate quality, compare applicants, identify missing skills, and make faster shortlisting decisions while keeping the final judgment with the human recruiter.

The project also demonstrates how modern semantic AI can be applied responsibly in a student project. The system does not claim to replace recruiters; it assists them with structured evidence, ranking, and analytics so that resume screening becomes faster and more consistent.

The final contribution of the project is not only the ranking table but the complete decision-support flow around it. A recruiter can define a job, upload multiple resumes, compare applicants against the same requirement, view score distributions, inspect skill gaps, change candidate status, and export selected candidates. This makes the application suitable for academic demonstration because every result visible on the dashboard can be traced back to a real uploaded resume and a specific processing module.

ResumeIQ also shows that a useful AI application does not always require custom model training or a large external dataset. By using an existing sentence-transformer model, careful preprocessing, transparent scoring formulas, and clear dashboard design, the system achieves semantic screening in a form that students can explain during viva. The project therefore balances practical usefulness with educational clarity.

This part contributes to the complete understanding of ResumeIQ by connecting implementation details with recruiter usage. The system is not described only as code; it is described as a practical process that moves from input documents to ranked decisions.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.

10.2 References

The project refers to official and widely used documentation for Flask, SQLite, NLTK, Sentence Transformers, pdfplumber, python-docx, Chart.js, TailwindCSS, NumPy, scikit-learn, and ReportLab. Conceptual references include recruitment screening workflows, text preprocessing, semantic embeddings, cosine similarity, skill extraction, ranking systems, and recruiter analytics.

The design of this report follows a standard academic project documentation structure with cover page, certificates, abstract, acknowledgement, table of contents, introduction, requirement analysis, system design, methodology, implementation, testing, results, user guide, limitations, future scope, conclusion, and references. Content is written specifically for the ResumeIQ implementation.

Important technology references include Flask documentation for routes and templates, SQLite documentation for local relational storage, NLTK documentation for preprocessing resources, Sentence Transformers documentation for all-MiniLM-L6-v2 embeddings, pdfplumber documentation for PDF extraction, python-docx documentation for DOCX extraction, Chart.js documentation for browser charts, TailwindCSS documentation for styling, and ReportLab documentation for PDF generation.

For implementation study, Flask is used as the primary web framework because it keeps the routing and template flow simple. SQLite is referenced as the storage layer because it provides local persistence without requiring a separate database server. NLTK is referenced for tokenization, stopword removal, and lemmatization. Sentence Transformers is referenced for generating dense semantic vectors, and cosine similarity is used to compare those vectors in a mathematically explainable way.

For document handling, pdfplumber and python-docx are used as practical parsing references because resumes are commonly submitted as PDF and DOCX files. For the recruiter interface, TailwindCSS is used for responsive styling and Chart.js is used for dashboard charts. These references together support a system that is simple enough for student-level explanation while still demonstrating a modern AI-assisted recruitment workflow.

This part contributes to the complete understanding of ResumeIQ by connecting implementation details with recruiter usage. The system is not described only as code; it is described as a practical process that moves from input documents to ranked decisions.

For academic evaluation, this point is important because it shows that ResumeIQ was not built as a single isolated script. The project connects user interaction, document processing, NLP, semantic similarity, database storage, and visual analytics into a complete workflow. Each part has a measurable effect on the final recruiter dashboard.